Express.js, MongoDB & Mongoose

Complete Lecture Notes in Simple Language

Topics from the lecture slides, each with a clear definition, easy explanation and one simple example

1. How the Internet Works (DNS)	2. What is Express?
3. Express + Mongoose + MongoDB Architecture	4. Create an Application
5. Port & Starting a Server	6. Middleware
7. Routes for GET, POST, PUT, DELETE	8. Mounting (Routers)
9. What is MongoDB?	10. What is Mongoose?
11. What is ODM?	12. What is Nodemon?
13. Optimal Folder Structure	14. What is MVC?
15. Different Types of Routes (Route Params)	16. How to Export (module.exports)
17. Schema, Models, Documents & Collections	18. Controllers
19. HTTP Response Status Codes	20. The ref Option in Mongoose Schema

How to read these notes: every topic has (1) a Definition box, (2) a simple explanation in bullet points, and (3) one small example. The last pages give a one-page recap and a comparison with Java / Spring Boot.

1. How the Internet Works (DNS)

Definition: When you type a website name (like www.google.com), your computer cannot use the name directly. Computers talk using **IP addresses** (numbers like 216.58.215.78). **DNS (Domain Name System)** is the system that converts a human-friendly domain name into an IP address. A **DNS Resolver** does this job on your behalf by asking a chain of DNS servers.

Understand it simply

Think of DNS as the **phone book of the internet**: you know the person's name (domain), but to call them you need the phone number (IP address).

The 9 steps shown in the slide

Step	What happens
1	The END USER types **www.google.com** in the browser.
2	The browser sends this name to the **DNS Resolver**.
3	The resolver asks the **DNS Root name server**. The root replies: "Go to the name server for .com (TLD)."
4	The resolver asks the **.com TLD name server**. It replies: "Go to the Route 53 name server."
5	The resolver asks **Google's Route 53 name server** for www.google.com.
6	Route 53 replies with the IP address **216.58.215.78**.
7	The resolver sends the IP address back to the browser.
8	The browser sends an HTTPS request to the **web server** at that IP address.
9	The web server returns the **web page**, and the browser displays it.

Simple example

You type **www.google.com**. Without DNS you would have to type 216.58.215.78 every time.

```
Browser  -> Resolver : "What is the IP of www.google.com?"
Resolver -> Root     : "Who knows .com?"          -> "Ask the .com TLD server"
Resolver -> .com TLD : "Who knows google.com?"    -> "Ask Route 53"
Resolver -> Route 53 : "IP of www.google.com?"    -> "216.58.215.78"
Browser  -> 216.58.215.78 : GET https://www.google.com   (page comes back)
```

Remember: TLD = Top Level Domain (.com, .org, .in). Steps 1-7 are only about *finding* the address; steps 8-9 are the actual web page request.

2. What is Express?

Definition: Express is a back-end web application framework for **Node.js**. It gives ready-made tools to build server-side applications that handle HTTP requests and responses, connect to databases and do other tasks. It is flexible and modular, so you can add **middleware** and customise the app.

Understand it simply

- **Node.js** lets you run JavaScript on the server, but writing a full server with plain Node is long and repetitive.
- **Express** sits on top of Node.js and makes it short: routing (which URL does what), reading request data, sending responses.
- It is **modular**: you only add the parts you need (e.g. logging, authentication, JSON parsing).
- It is the 'E' in the **MERN** stack (MongoDB, Express, React, Node).

Simple example

A tiny server that sends a message when someone visits the home page:

```
const express = require('express');
const app = express();

app.get('/', (req, res) => {
  res.send('Welcome to my server!');
});

app.listen(3000);
```

Remember: Express = Node.js + easy routing + middleware. It is the back-end (server) part of MERN.

3. Express + Mongoose + MongoDB Architecture

Definition: This diagram shows how a full back-end is connected: **clients** (mobile app or website) send requests to the **API** built with Express.js; the API talks to the database through **Mongoose** and the **Mongo Driver**, which connect to **MongoDB** over TCP/IP.

Understand it simply

- **Clients:** React Native (Android/iOS) apps and WEB browsers (Chrome, Firefox, Opera, Safari) - they only send requests and show results.
- **API (Express.js):** receives requests, applies business rules, decides what to do.
- **Mongoose + Mongo Driver:** translate your JavaScript objects into MongoDB commands (the arrow in the diagram works both ways: request goes in, data comes back).
- **TCP/IP:** the network connection between your server and the database server.
- **MongoDB:** stores the data in a database that contains collections (the small cylinders).

Simple example

Flow when a mobile user opens 'My Profile':

```
React Native app --GET /profile/5--> Express API
Express API --> Mongoose (User.findById(5)) --> Mongo Driver
Mongo Driver --TCP/IP--> MongoDB
MongoDB --> user data --> back through the same path --> app shows the profile
```

Remember: One back-end API can serve many clients (mobile + web) at the same time.

4. Create an Application

Definition: Creating an Express application means importing the Express library, creating an **app object**, defining at least one **route**, and telling the app to **listen** on a port so it can start receiving requests.

Understand it simply

- Line 1: `require('express')` imports the Express library.
- Line 2: `express()` creates the application (the 'app' object).
- Line 3: `port = 3000` is the door number where the server will wait.
- Lines 5-7: `app.get('/')` says: when someone opens the home page (/), run this function. `req` = request (what the user sent), `res` = response (what we send back).
- Lines 9-11: `app.listen` starts the server and prints a message in the console.

Simple example

The exact code from the slide (install first with `npm install express`, run with `node app.js`). Then open http://localhost:3000 to see 'Hello World!'.

```
const express = require('express')
const app = express()
const port = 3000

app.get('/', (req, res) => {
  res.send('Hello World!')
})

app.listen(port, () => {
  console.log(`Example app listening on port ${port}`)
})
```

Remember: Every Express app = import -> create app -> add routes -> listen on a port.

5. Port & Starting a Server

Definition: A **port** is a number that identifies one specific communication endpoint on a computer. When an Express server starts, it must be told which port to **listen** on so that clients can connect and communicate with it.

Understand it simply

- **Why do we need a port?** One computer can run many server programs at once (a website, a database, an API). The port number makes sure each request reaches the *correct* program.
- **Analogy:** The computer is an apartment building, and each port is a flat number. The IP address gets you to the building; the port gets you to the right flat.
- The slide says Express uses 3000 by default. (Precisely: in the code you choose the number yourself in `app.listen()`; 3000 is just the common convention.)
- You can use any **free** port. If the port is already used by another program, you get an 'address already in use' error.
- Browsers use 80 for HTTP and 443 for HTTPS automatically, so you never type them for normal websites.

Simple example

Run two apps on one computer using two different ports:

```
// api.js
app.listen(3000, () => console.log('API on http://localhost:3000'));

// admin.js
app.listen(4000, () => console.log('Admin on http://localhost:4000'));
```

Remember: Format: http://localhost:PORT. Different apps on the same machine need different ports.

6. Middleware

Definition: Middleware is a function that runs **between** the incoming request and the final response. It can read or modify the request/response objects, call the **next** middleware in the chain, or end the cycle by sending a response.

Understand it simply

- Think of middleware as **security checks at an airport**: ticket check -> baggage scan -> boarding. A request passes through each step in order before reaching the destination (your route).
- **Logging:** record details of every request and response.
- **Authentication:** check who the user is and whether they may access a resource, or redirect to a login page.
- **Parsing:** read the request body (JSON, XML, form data) so you can use it in `req.body`.
- **Error handling:** catch errors that happen during the request/response cycle.
- Added with `app.use()`. They run in the **same order** in which you add them. Calling `next()` passes control to the next one.

Simple example

A logger middleware, a JSON parser and a login check:

```
const express = require('express');
const app = express();

app.use(express.json());              // parsing: reads JSON body

app.use((req, res, next) => {         // logging
  console.log(req.method, req.url);
  next();                             // go to the next step
});

function checkLogin(req, res, next) { // authentication
  if (!req.headers.token) return res.status(401).send('Please login');
  next();
}

app.get('/dashboard', checkLogin, (req, res) => res.send('Welcome!'));
```

Remember: No next() = the request gets stuck (unless you already sent a response). Order of app.use() matters.

7. Routes for GET, POST, PUT and DELETE

Definition: A **route** connects an **HTTP method** + a **URL path** to a function. The four main methods map to CRUD: **GET** = Read, **POST** = Create, **PUT** = Update, **DELETE** = Delete.

Understand it simply

- **GET** - ask the server for data (open a page, list users).
- **POST** - send new data to the server (sign up, add a product).
- **PUT** - update existing data (edit profile).
- **DELETE** - remove data (delete account).
- In Express the method name is the function name: `app.get()`, `app.post()`, `app.put()`, `app.delete()`.

Simple example

All four routes exactly as shown in the slide:

```
// GET on homepage
app.get('/', (req, res) => {
  res.send('Hello World!')
})

// POST on root route (/)
app.post('/', (req, res) => {
  res.send('Got a POST request')
})

// PUT on /user
app.put('/user', (req, res) => {
  res.send('Got a PUT request at /user')
})

// DELETE on /user
app.delete('/user', (req, res) => {
  res.send('Got a DELETE request at /user')
})
```

Remember: Same URL (/user) can do different things depending on the method (PUT vs DELETE).

8. Mounting (Routers)

Definition: Mounting means attaching middleware or a sub-application (a **Router**) to a specific path of the main Express app using `app.use()`. Each router handles only the requests that start with its path.

Understand it simply

- Instead of writing every route in one huge file, you create small router files: one for users, one for products, and so on.
- `app.use('/users', userRouter)` means: every request starting with **/users** goes to **userRouter**.
- `app.use('/products', productRouter)` means: every request starting with **/products** goes to **productRouter**.
- Benefit: the code is **organised, easy to maintain and easy to scale**. You can add or remove a module without disturbing the rest.

Simple example

The slide's code plus what is inside a router file:

```
// app.js
const express = require('express')
const app = express()

const userRouter = require('./routes/user')
app.use('/users', userRouter)

const productRouter = require('./routes/product')
app.use('/products', productRouter)

// routes/user.js
const router = require('express').Router();
router.get('/', (req, res) => res.send('All users'));       // GET /users
router.get('/:id', (req, res) => res.send('One user'));     // GET /users/5
module.exports = router;
```

Remember: Inside the router you write '/' - the mount path (/users) is added automatically in front.

9. What is MongoDB?

Definition: MongoDB is a **NoSQL, document-oriented database**. Instead of tables and rows, it stores data as **JSON-like documents** with a flexible (dynamic) schema. It is the database layer of the **MERN** stack (MongoDB, Express.js, React, Node.js).

Understand it simply

- **NoSQL** = not only SQL. No fixed tables; data is kept as documents.
- **Flexible:** two documents in the same collection can have different fields.
- **Scalable:** handles large amounts of data well.
- Easy to work with complex, nested data (e.g. a user with a list of addresses inside the same document).
- In MERN, Node.js uses the **Mongoose** library (which uses the Mongo Driver) to store and fetch data from MongoDB - as in the slide's diagram: Node -> Mongoose -> Mongo Driver -> MongoDB.

Simple example

The same student in SQL vs MongoDB:

```
SQL table 'students':
 id | name  | city
 1  | Ravi  | Delhi

MongoDB document in collection 'students':
{
  "_id": "64a1...",
  "name": "Ravi",
  "city": "Delhi",
  "skills": ["Java", "React"]     // extra field, no table change needed
}
```

Remember: Table -> Collection, Row -> Document, Column -> Field.

10. What is Mongoose?

Definition: Mongoose is an **Object Data Modeling (ODM)** library for MongoDB and Node.js. It gives a higher-level layer to work with MongoDB, where you define **schemas** (data structure + validation rules) that map to MongoDB collections.

Understand it simply

- MongoDB itself is very flexible (accepts anything). Mongoose adds **structure and rules** so your data stays clean and reliable.
- **Schema definition:** define fields, data types, default values and validations.
- **Querying and manipulation:** simple methods such as `find()`, `findOne()`, `save()`, `update()`.
- **Middleware (hooks):** functions that run before/after operations - e.g. validate data, encrypt a password before saving, send a notification.
- **Connection management:** handles connection pooling and reconnects if the network fails.
- **Plugins:** add extra features such as full-text search or geospatial (location) queries.

Simple example

Connect, define a schema, and save one user:

```
const mongoose = require('mongoose');
mongoose.connect('mongodb://localhost:27017/myapp');

const userSchema = new mongoose.Schema({
  name:  { type: String, required: true },
  email: { type: String, unique: true },
  age:   Number
});
const User = mongoose.model('User', userSchema);

await User.create({ name: 'Aman', email: 'aman@mail.com', age: 22 });
const users = await User.find({ age: { $gt: 18 } });
```

Remember: Mongoose = rules + easy methods on top of MongoDB.

11. What is ODM?

Definition: An **ODM (Object Data Modeling) library** is a tool that adds an abstraction layer on top of a database so that you work with your data using **objects** (object-oriented style) instead of writing raw database queries.

Understand it simply

- It is mostly used with **NoSQL** databases like MongoDB, whose data is naturally shaped like objects.
- You work with data the way your programming language does: objects, properties and methods.
- It makes it easier to **map data to objects**, perform **validation** and handle **relationships** between data.
- Examples: **Mongoose** for MongoDB, an ODM for Couchbase, Doctrine for PHP.
- Result: higher productivity, fewer errors and more maintainable code.
- **ODM vs ORM:** ODM is for document databases (MongoDB); ORM (Object Relational Mapping, e.g. Hibernate) is for relational/SQL databases.

Simple example

Same task without and with an ODM:

```
// Without ODM (raw driver)
db.collection('users').insertOne({ name: 'Aman', age: 22 });

// With ODM (Mongoose) - you work with an object
const user = new User({ name: 'Aman', age: 22 });
await user.save();
```

Remember: ODM = object-style access to a document database. ORM = same idea for SQL databases.

12. What is Nodemon?

Definition: Nodemon is a development tool for Node.js that **watches your project files** and **automatically restarts** the application whenever it detects a change.

Understand it simply

- Without it: change code -> stop server (Ctrl+C) -> start again (node app.js). Every single time.
- With it: just save the file and the server restarts by itself.
- By default it watches `.js`, `.json` and `.html` files; it can be configured to watch other file types.
- Install it as a **development dependency** (only needed while developing), then run `nodemon` instead of `node`.

Simple example

Install and use:

```
npm install --save-dev nodemon

# instead of:  node app.js
nodemon app.js

# or add a script in package.json:
"scripts": {
  "dev": "nodemon app.js"
}
# then run:  npm run dev
```

Remember: Nodemon is for development only. It is similar to Spring Boot DevTools auto-restart.

13. Optimal Folder Structure

Definition: An **optimal folder structure** is a clean way to organise project files so each folder has one responsibility. The slide shows the flow: **Routes -> Controllers -> Models -> Database**, with **Views** used by controllers to build the HTML response.

Understand it simply

- **Routes:** receive the HTTP request and forward it to the correct controller.
- **Controllers:** contain the logic; they read/write data through models and send the HTTP response.
- **Models:** talk to the database (read/write data).
- **Views (templates):** the HTML templates that controllers fill with data (used when the server returns HTML pages; for REST APIs the response is usually JSON).
- **Database:** where the data is finally stored.

Simple example

A typical project layout:

```
my-app/
  |-- app.js              (starts the server, mounts routes)
  |-- package.json
  |-- routes/
  |     |-- user.js       (URL -> controller function)
  |-- controllers/
  |     |-- userController.js   (logic)
  |-- models/
  |     |-- User.js       (Mongoose schema + model)
  |-- views/              (optional templates)
  |-- middleware/         (optional: auth, logger)
```

Remember: Request path: Route -> Controller -> Model -> Database, and the response travels back through the controller.

14. What is MVC?

Definition: MVC (Model-View-Controller) is a software architecture pattern that separates an application into three connected parts: **Model** (data and business logic), **View** (user interface) and **Controller** (the middle-man).

Understand it simply

- **Model:** represents the data and business rules - database operations, data validation, application rules.
- **View:** what the user sees - HTML templates, CSS, JavaScript that handles user interaction. It renders data in an easy-to-understand way.
- **Controller:** receives user requests, asks the model for data, passes the data to the view, and updates the model when the user sends input.
- **Why use MVC?** A change in one part does not break the others, so the code is modular, maintainable, scalable and testable.
- Used in Ruby on Rails, Django, Laravel, Express.js (and Spring MVC in Java).
- **Analogy - Restaurant:** Customer = user, Waiter = Controller, Kitchen/Stock = Model, Plated dish = View.

Simple example

'Show user 5' in MVC:

```
1. Browser requests  GET /users/5
2. Controller  -> asks Model: User.findById(5)
3. Model       -> fetches the data from the database
4. Controller  -> gives the data to the View (or sends JSON)
5. View        -> displays "Name: Ravi, City: Delhi"
```

Remember: MVC separates concerns: data (Model), display (View), logic/flow (Controller).

15. Different Types of Routes (Route Parameters)

Definition: A **route parameter** is a variable part of a URL, written with a colon like `:id`. Express captures its value and gives it to your code in `req.params`.

Understand it simply

- In `/car/:id`, the **:id** part can be any value - it stands for a unique identifier of one car.
- If the request is `/car/123`, Express captures **"123"** as the id.
- In server code you read it using `req.params.id` (req = the request object).
- Then you use that value to query the database or do some action for that specific car.
- The slide's right side shows a typical REST API (a pet store): **GET** /pet/{petId} find pet by ID, **PUT** /pet update a pet, **DELETE** /pet/{petId} delete a pet, **POST** /pet/{petId}/uploadImage upload an image.

Simple example

Get, update and delete one car by its id:

```
app.get('/car/:id', (req, res) => {
  const id = req.params.id;          // "123" for /car/123
  res.send('You asked for car number ' + id);
});

app.delete('/car/:id', (req, res) => {
  res.send('Deleted car ' + req.params.id);
});

// With database:  const car = await Car.findById(req.params.id);
```

Remember: Route parameter = part of the path (/car/123). Do not confuse with query string (/car?color=red), which is read from req.query.

16. How to Export

Definition: Exporting means making a function or object from one file (module) available so that other files can **import** it with `require()`. In Node.js/Express this is done with `module.exports` or `exports`.

Understand it simply

- Each file is a separate module; things inside it are private unless you export them. (Think: *public* vs *private* in Java.)
- **1. Export one thing directly:** `module.exports = myFunction;` or `module.exports = myObject;`
- **2. Export multiple things:** attach them to `exports`: `exports.myFunction1 = myFunction1;` `exports.myFunction2 = myFunction2;` (same for objects).
- **3. Export a 'default' function/object:** also written as `module.exports = myFunction;` - when a file exports just one main thing, that is its default.
- Which style to use depends on the use case and your preference.

Simple example

Export from one file, import in another:

```
// math.js  - exporting multiple functions
exports.add = (a, b) => a + b;
exports.sub = (a, b) => a - b;

// logger.js - exporting one default function
module.exports = function log(msg) { console.log(msg); };

// app.js - importing
const { add, sub } = require('./math');
const log = require('./logger');
log(add(2, 3));      // prints 5
```

Remember: Do not mix: if you assign module.exports = ..., earlier exports.xxx assignments in that file are ignored.

17. Schema, Models, Documents & Collections

Definition: In Mongoose/MongoDB: a **Schema** is the blueprint of a document, a **Model** is built from the schema and used for CRUD operations, a **Document** is one record, and a **Collection** is a group of related documents.

Understand it simply

- **Schema:** blueprint describing the fields, data types, default values and validation rules. Keeps data consistent.
- **Model:** created from a schema; it is your interface to the collection - you use it to Create, Read, Update, Delete.
- **Document:** an instance of a model - one single record (e.g. one student with name and phone).
- **Collection:** a group of documents; similar to a **table** in a relational database.
- The slide's picture also shows: a **Path** is one field in a schema, and it can have several properties, e.g. `firstname: { type: String, required: true }`.
- **Analogy:** Schema = the blank admission form design, Model = the admission office, Document = one filled form, Collection = the file cabinet of all forms.

Simple example

One example showing all four:

```
const studentSchema = new mongoose.Schema({        // SCHEMA (blueprint)
  firstname: { type: String, required: true },     // each field = a PATH
  lastname:  String,
  telephone: String
});

const Student = mongoose.model('Student', studentSchema);  // MODEL
// -> MongoDB collection name will be 'students'  (COLLECTION)

const s = new Student({ firstname: 'Ronnie', lastname: 'Barker',
                        telephone: '555-1234' });          // DOCUMENT
await s.save();
```

Remember: Schema -> Model -> Documents -> stored together in a Collection.

18. Controllers

Definition: Controllers are the functions that contain the application's **business logic** and **handle user requests**. They receive the request, use models/services to do the work and send back the response.

Understand it simply

- **Separation of concerns:** logic lives in controllers, not mixed with models, views or routes - easier to manage and fewer bugs.
- **Handling user requests:** e.g. to create a resource, the controller validates the data, creates it using the model, and sends a response.
- **Reusability:** one controller function can be used by multiple routes, which reduces duplicate code and makes testing easier.
- **Error handling:** catch errors, log them, and send suitable error responses to keep the app stable and reliable.
- **Java comparison:** a controller is like a method in a Spring `@RestController`.

Simple example

Controller file + route that uses it:

```
// controllers/userController.js
const User = require('../models/User');

exports.createUser = async (req, res) => {
  try {
    if (!req.body.name) return res.status(400).json({ error: 'Name required' });
    const user = await User.create(req.body);
    res.status(201).json(user);
  } catch (err) {
    console.error(err);
    res.status(500).json({ error: 'Server error' });
  }
};

// routes/user.js
const { createUser } = require('../controllers/userController');
router.post('/', createUser);
```

Remember: Route = 'which URL'. Controller = 'what to do'. Model = 'how data is stored'.

19. HTTP Response Status Codes

Definition: HTTP response status codes are 3-digit numbers sent by the server that tell the client whether a particular HTTP request was **successfully completed** or what went wrong. They are grouped into five classes.

Understand it simply

- The first digit tells the class. Learn the five classes first, then the common codes.

Class	Range	Meaning	Common codes
1. Informational	100 - 199	Request received, continuing	100 Continue
2. Successful	200 - 299	Request worked	200 OK, 201 Created, 204 No Content
3. Redirection	300 - 399	Go somewhere else	301 Moved Permanently, 304 Not Modified
4. Client error	400 - 499	The client made a mistake	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found
5. Server error	500 - 599	The server failed	500 Internal Server Error, 503 Service Unavailable

Simple example

The five classes (from the slide) and the codes you will use most (added for clarity):

```
app.get('/car/:id', async (req, res) => {
  const car = await Car.findById(req.params.id);
  if (!car) return res.status(404).send('Car not found');   // client asked for missing data
  res.status(200).json(car);                                // success
});

app.post('/car', async (req, res) => {
  const car = await Car.create(req.body);
  res.status(201).json(car);                                // 201 = new resource created
});
```

Remember: 2xx = success, 4xx = your (client) fault, 5xx = server's fault. Easy memory trick: 4 = 'you', 5 = 'us'.

20. The "ref" Option in Mongoose Schema

Definition: The **ref** option is used inside a Mongoose schema to create a **relationship between two models (collections)**. It tells Mongoose which model a field's **ObjectId** points to. Mongoose can then **populate** (fetch and fill in) the referenced document automatically.

Understand it simply

- Instead of copying a full author inside every book, you store just the author's **ObjectId** (like a foreign key in SQL).
- In the slide: **authorSchema** has `books` - an array of ObjectIds referencing the **Book** model; **bookSchema** has `author` - one ObjectId referencing the **Author** model.
- This makes it easy to find all books by an author, or the author of a book.
- **populate():** when you query a book and ask to populate 'author', Mongoose automatically fetches the Author document and replaces the id with the full author data.
- Without populate you only see the id; with populate you see the complete related object.

Simple example

The slide's schemas, and how to use populate:

```
const mongoose = require('mongoose');

const authorSchema = new mongoose.Schema({
  name: String,
  age: Number,
  books: [{ type: mongoose.Schema.Types.ObjectId, ref: 'Book' }]
});

const bookSchema = new mongoose.Schema({
  title: String,
  author: { type: mongoose.Schema.Types.ObjectId, ref: 'Author' }
});

const Author = mongoose.model('Author', authorSchema);
const Book = mongoose.model('Book', bookSchema);

// Using populate:
const book = await Book.findOne({ title: 'Wings of Fire' }).populate('author');
console.log(book.author.name);   // full author object, not just an id
```

Remember: ref = which model the id belongs to. populate() = replace the id with the real document. It is similar to a JOIN in SQL / @ManyToOne in JPA.

. One-Page Recap

Topic	One-line meaning
DNS	Converts domain name to IP address.
Express	Node.js framework to build back-end servers and APIs.
Port / listen	Door number where the server waits for requests.
Middleware	Function that runs between request and response (log, auth, parse, errors).
Routes	Method + URL -> function (GET read, POST create, PUT update, DELETE delete).
Mounting	Attach a router to a path with app.use('/users', userRouter).
MongoDB	NoSQL database storing JSON-like documents.
Mongoose / ODM	Library giving schemas, validation and easy object-style queries for MongoDB.
Nodemon	Auto-restarts the server on file changes (development).
Folder structure / MVC	Routes -> Controllers -> Models -> Database; keep concerns separate.
Route params	/car/:id gives req.params.id.
Export	module.exports / exports to share code between files.
Schema, Model, Document, Collection	Blueprint, tool for CRUD, one record, group of records.
Controller	Holds business logic and error handling for a request.
Status codes	2xx success, 3xx redirect, 4xx client error, 5xx server error.
ref + populate	Link collections using ObjectId and fetch related data.

Express/MongoDB vs Java / Spring Boot (quick comparison)

Node / Express / Mongoose	Java / Spring Boot equivalent
Express app, app.get('/path')	Spring Boot app, @GetMapping('/path')
Router + app.use('/users', router)	@RestController + @RequestMapping('/users')
Middleware (app.use)	Servlet Filter / HandlerInterceptor / Spring Security filter chain
Controller function	Controller class method
Mongoose Model / Schema	JPA Entity + Repository (Spring Data MongoDB for MongoDB)
ref + populate()	@ManyToOne / @OneToMany with JOIN fetch
req.params.id	@PathVariable
req.body (express.json())	@RequestBody
res.status(404)	ResponseEntity.status(HttpStatus.NOT_FOUND)
Nodemon	Spring Boot DevTools
module.exports / require	public classes / import